PROJECT ACTIVITY REPORT

Activity 1.2: Install Required Libraries and Set Up Virtual Environment

PROJECT TITLE: **Multimodal Attendance & Engagement Agent**

Objective

The objective of this activity was to set up a dedicated Python development environment for the Multimodal Attendance & Engagement Agent project. A virtual environment was created to isolate project dependencies from the system-wide Python installation, preventing version conflicts and ensuring a stable, portable, and reproducible development environment. All required libraries were installed using the `requirements.txt` file and verified through successful module imports and backend server execution.

Introduction

SECTION 01

Modern Artificial Intelligence (AI) applications rely on numerous third-party libraries for computer vision, natural language processing, speech recognition, deep learning, and backend development. Managing these dependencies efficiently is essential for maintaining software stability and ensuring that the application runs consistently across different systems.

To address this requirement, a dedicated Python virtual environment was created for the project. The virtual environment isolates project-specific packages from global Python packages, making the application easier to develop, test, and deploy. This approach follows professional software engineering practices widely adopted in AI and cloud-based development.

Approach

SECTION 02

The project environment was configured using Python's built-in `venv` module. A separate virtual environment was created specifically for the project to avoid dependency conflicts with other Python applications.

After activating the virtual environment, all project dependencies were installed using a single `requirements.txt` file. This file contains a list of all libraries required by the project, ensuring that every developer working on the project installs identical package versions.

Finally, the installation was validated by importing essential libraries and launching the backend server successfully. This confirmed that the environment was configured correctly and ready for further development.

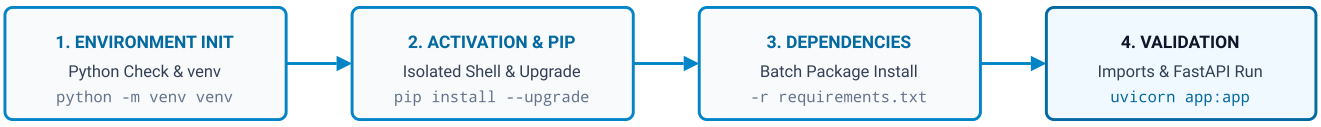


Figure 1: Four-stage workflow for virtual environment isolation, dependency installation, and server validation.

Process

SECTION 03

STEP 1: Verify Python Installation

The first step involved verifying that Python was installed correctly and that the installed version was compatible with the project requirements.

COMMAND: `python --version`

This command displays the currently installed Python version.

STEP 2: Create a Virtual Environment

A dedicated virtual environment named `venv` was created inside the project directory.

COMMAND: `python -m venv venv`

This command generates an isolated environment containing its own Python interpreter and package directory.

STEP 3: Activate the Virtual Environment

Before installing any libraries, the virtual environment was activated.

> **Windows Command Prompt:** `venv\Scripts\activate`
> **Windows PowerShell:** `.\venv\Scripts\Activate.ps1`
> **Linux/macOS:** `source venv/bin/activate`

After activation, the terminal prompt displayed `(venv)`, indicating that the virtual environment was active.

STEP 4: Upgrade pip

The package manager (`pip`) was upgraded to the latest version before installing project dependencies.

COMMAND: `python -m pip install --upgrade pip`

Keeping `pip` updated improves compatibility with modern Python packages and reduces installation errors.

STEP 5: Install Required Libraries

All required project dependencies were installed using the `requirements.txt` file.

COMMAND: `pip install -r requirements.txt`

The installation process automatically downloaded and installed every library required for the project. These libraries support:

- Computer Vision
- Face Detection and Recognition
- Speech Processing
- Natural Language Processing
- Deep Learning Models
- Data Processing
- Backend API Development
- Database Connectivity

STEP 6: Verify Installed Packages

After installation, all installed packages were verified.

COMMAND: `pip list`

STEP 7: Verify Library Imports

A simple Python script was created to ensure that all essential libraries were imported successfully. Example:

```
      import cv2 import numpy import pandas import torch import transformers import
fastapi print("All required libraries imported successfully.")
```

If the script executes without any errors, it confirms that the installation was successful.

STEP 8: Launch the Backend Server

The backend server was started to verify that all dependencies were functioning correctly.

EXAMPLE COMMAND: `uvicorn app:app --reload`

A successful server startup indicated that the development environment had been configured properly.

Technical Details

SECTION 04

The project uses the following technologies for environment setup and dependency management.

COMPONENT	TECHNOLOGY USED
Programming Language	Python 3.x
Virtual Environment	Python venv
Package Manager	pip
Dependency File	requirements.txt
Backend Framework	FastAPI
AI Libraries	PyTorch, Transformers
Computer Vision	OpenCV
Data Processing	NumPy, Pandas

Advantages of Using a Virtual Environment

SECTION 05

Using a virtual environment provides several benefits:

- Prevents conflicts between different Python projects.
- Keeps project dependencies isolated from the system installation.
- Ensures reproducibility across multiple development systems.
- Simplifies deployment and collaboration.
- Allows different projects to use different library versions.
- Improves project maintainability and stability.
- Follows industry-standard software engineering practices.

Validation

SECTION 06

The environment setup was validated through the following checks:

✓ Python installation verified successfully.
✓ Virtual environment created successfully.
✓ Virtual environment activated successfully.
✓ Installed packages verified using pip list.
✓ Core libraries imported successfully.
✓ Backend server launched successfully.